**2. Schema-constrained extraction**

*A downstream legal database needs structured JSON pulled from vendor contracts.*

Bad prompt: "Here's a 40-page vendor contract. Pull out the important stuff."

**System Prompt:**

You are a legal contract extraction assistant. Extract only explicitly stated information. Do not guess or infer. Return valid JSON only.

**User Prompt:**

Extract these fields from the vendor contract:

- Vendor name
- Customer name
- Effective date
- Contract value
- Payment terms
- Renewal terms
- Termination notice
- Governing law
- Liability cap

Use this schema:

```
{
 "vendor_name": null,
 "customer_name": null,
 "effective_date": null,
 "contract_value": null,
 "payment_terms": null,
 "renewal_terms": null,
 "termination_notice": null,
 "governing_law": null,
 "liability_cap": null
}
```

Use `null` when information is missing. Do not add extra fields.

**3. RAG with grounding and citation control**

*An internal HR chatbot answers employee questions from a set of policy PDFs, some of which are outdated and contradict newer ones.*

Bad prompt: "Using the attached documents, answer the employee's question."

Using the provided HR policy documents, answer the employee's question using only the information in the documents. Check the policy dates and prefer the latest applicable policy when documents conflict. Do not guess or use outside knowledge. Explain the answer clearly and provide the relevant document, section, and page as citations. If the documents do not contain enough information, say so.

**4. LLM-as-judge / rubric design**

*An LLM pre-screens 200 scholarship essays before human review.*

Bad prompt: "Grade this essay from 1 to 10."

Evaluate the scholarship essay using the provided rubric. Score each criterion from 1–5 and give a brief evidence-based reason for every score. Evaluate only the stated criteria and do not make assumptions about the applicant. Calculate the total score, identify key strengths and weaknesses, and flag the essay for human review when the evidence is unclear or borderline.

**5. Code generation with full specification**

*Generating a utility function for a shared production library.*

Bad prompt: "Write me a function that finds duplicates in a list, make it good."

Write a production-ready Python 3.11+ function called `find_duplicates(items)`. Return each duplicated value only once, preserving the order of its first appearance. Do not modify the input and target O(n) average time complexity. Handle empty lists, no duplicates, and unhashable values appropriately. Include type hints, a clear docstring, error handling, and unit tests covering normal and edge cases.

**6. Safety-calibrated output in a high-stakes domain**

*A tool gives clinicians an AI-generated first read on dermoscopic images, meant to sit alongside — not replace — their judgment.*

Bad prompt: "Look at this skin lesion photo and tell me if it's cancer."

Review the attached dermoscopic image as a clinical decision-support aid. Describe only the visible features you can reasonably identify, mention any image-quality limitations, and explain possible clinical considerations using uncertainty-aware language. Do not give a definitive diagnosis or state that the lesion is definitely cancerous or benign. Clearly explain the limitations of image-based assessment and emphasize that the final decision must be made by a qualified clinician.